

Zero Click Attacks

1 What Is a Zero-Click Attack?

A zero-click attack is a type of cyberattack that requires no interaction from the victim. The attacker exploits software vulnerabilities to gain access to devices without the user clicking links, downloading files, or taking any action. Medium Common entry points include messaging platforms, VoIP services, push notification handlers, image and video parsers, and other background services that automatically receive and process data. These vulnerabilities often stem from memory corruption bugs, parser logic errors, or sandbox escape conditions within trusted system components. From a broader cybersecurity perspective, zero-click attacks reflect a shift away from social engineering toward exploitation of automated workflows that users and systems inherently trust.

2 Scale & Threat Landscape

The sophistication and scale of zero-click attacks in 2025 represent a paradigm shift. Google's Threat Intelligence Group documented 75 zero-day vulnerabilities actively exploited in 2024, with the trend accelerating into 2025 as attackers pivoted toward enterprise infrastructure. In the first half of 2025 alone, more than 21,500 CVEs were newly disclosed, representing an 18% increase over the previous year. More alarmingly, the "time to exploit" window collapsed to an average of just five days in 2024, down from 32 days in previous years, rendering traditional monthly patch cycles dangerously obsolete [ADD REFERENCE HERE](#). Cyber Security News Kaspersky data shows 29% more attacks targeting smartphone users in the first half of 2025 than in the same period of 2024. The U.K. National Cyber Security Centre recently warned that roughly 100 countries now possess commercial spyware capabilities capable of compromising smartphones and computers.

3 Typical Lifecycle of a Mobile Zero-Click Spyware Attack

Usually, mobile spyware campaigns such as Pegasus commonly follow four main phases.

- **Delivery:** The attacker sends specially crafted data to a service that automatically processes incoming content. This could be an iMessage containing a malicious PDF disguised as a GIF, a malicious PassKit attachment, a specially formed series of data packets sent to a device's Wi-Fi chipset, or a VoIP call that is never answered. The attacker may only need information such as the victim's phone number, account identifier, or Apple ID. The function of the spyware can be identified through forensic analysis of the infected device. Investigators analyze device logs, suspicious processes, network connections, databases, and traces left after infection. From these indicators, they can understand what the spyware accessed or attempted to access. In the case of Pegasus, this shows that the malware is used to monitor the victim's device and collect sensitive information from it.
- **Exploitation:** After the malicious data reaches the device, the targeted application or system processes it automatically. This triggers a vulnerability, such as memory **corruption bug** or logic flaw, without requiring the victim to click, open or approve anything. If the exploitation succeeds, attacker-controlled code may run with the privileges of the affected application or service. A memory corruption happens when a program incorrectly handles memory, for example by reading or writing outside the intended memory area. In zero-click attacks, crafted data such as image, PDG, or media file can trigger this bug automatically when the device processes the content. If the attacker controls the corrupted memory, the bug may lead to code execution. This is clearly shown in the cases of FORCEDENTRY, BLASTPASS and CVE-2019-3568 in section 4.1, CVE-2026-0006 in section 4.1.2, Apple Mail CVE-2020-9818/CVE-2020-9819 in section 4.2 and CVE-2026-40361 in section 4.2
- **Installation and Persistence:** Initial code execution may provide only limited access because mobile applications and services usually operate inside restricted environments or sandboxes. Attackers may therefore chain additional vulnerabilities to **escape the sandbox, escalate privileges, and execute a more advanced payload** such as spyware. Depending on the exploit chain, the attacker may gain access to sensitive data, communications, location information, the microphone, or the camera.
- **Evasion and Cleanup:** Sophisticated attackers may attempt to hide the compromise by deleting malicious messages, attachments, calendar entries, temporary files, or other traces. However, cleanup may not always be complete. Investigators may still find indirect evidence in device logs, crash reports, network activity, application databases, or records of unexpected service relaunches, and attachment storage locations such as **Library/SMS/Attachments**. These traces may include process crashes, suspicious network connections, delete database entries such as traces in **DataUsage.sqlite**, or unusual system logs. [1]

4 Domains of Implementation

4.1 Mobile Zero-Click Attacks Abusing Automatic Processing

The Pegasus / NSO Group Chronicles This is the most documented and technically sophisticated real-world use of zero-click attacks. Pegasus is a spyware developed by NSO Group that can infect a victim’s device without requiring the victim to click on anything. The spyware can be delivered through applications that automatically process incoming data, such that iMessage, WhatsApp, or other communication services. In these cases, the victim’s device may receive a malicious message, file, call, or attachment, and the vulnerable application processes it in the background. This allows the attack to be triggered silently, making Pegasus one of the most sophisticated examples of zero-click exploitation [2].

4.1.1 Messaging and Attachment Processing Attacks

- KISMET (2020): Zero-click exploit chain used to compromise iPhones running iOS 13.x through iMessage. It appeared to involve an invisible zero-click exploit in iMessage. In July 2020, KISMET worked against at least 13.5.1 and was able to compromise Apple’s then-latest iPhone 11. This means that the victim did not need to click a link, open an attachment, or interact with the message. The compromise happened because the iMessage-related services processed the malicious content automatically as part of their normal background operation. Once the specially crafted iMessage-related data reached the target iPhone, this automatic processing appears to have triggered the exploit chain and allowed the attack to continue without any visible action from the victim. Citizen Lab’s forensic analysis found unusual Apple/iCloud-related network activity. Before the phone contacted a Pegasus installation server, there was unusual Apple/iCloud-related network activity. The imagent process, which is linked to iMessage and FaceTime, also appeared in the spyware process chain. This suggests that KISMET began through Apple/iMessage-related background processing before Pegasus was delivered and installed on the device. The exact technical vulnerability used by KISMET was not publicly documented in detail. However, what is known is that the specifically crafted iMessage acted as the delivery vector: it delivered malicious data to a vulnerable iMessage-related processing path, which then allowed the exploit chain to execute. KISMET did not appear to work against iOS 14 and later versions, because iOS 14 introduced stronger protections. [3] One major protection introduced in iOS 14 was BlastDoor, a sandboxed service that isolates, parses, transcodes, and validates untrusted data arriving through Messages before it can reach more sensitive parts of the operating system. [4] iOS 14 also made exploitation harder by moving most untrusted iMessage parsing into the tightly sandboxed BlastDoor service,

which was written in Swift. It also made exploitation harder by improving shared-cache memory randomization and slowing down repeated exploit attempts after crashes through exponential throttling. These changes reduced the effectiveness of zero-click attacks because malicious iMessages became harder to exploit reliably and harder to repeat quickly if the exploit failed. [5]

- **FORCEDENTRY (2021)** : Discovered on the iPhone of a Saudi activist, it was a zero-day, zero-click exploit against Apple’s iMessage remarkable for its complexity and ability to bypass advanced security mitigations. Specifically, attackers sent what appeared to be a harmless GIF file, which was actually a manipulated PDF. A vulnerability in Apple’s image processing library (CoreGraphics) was exploited to escape the iMessage sandbox and execute arbitrary code [6, 7]. Google’s Project Zero assessed FORCEDENTRY as “one of the most technically sophisticated exploits we’ve ever seen.” By targeting iMessage, the NSO Group hackers needed only a phone number or Apple ID username to take aim and fire eavesdropping implants [8]. After successful exploitation, FORCEDENTRY allowed Pegasus spyware to be delivered and installed on the victim’s device. Once Pegasus was installed, the attacker could secretly monitor the iPhone and collect sensitive information such as messages, chats, emails, media, contacts, calls, and location data. Pegasus could also access device sensors such as the microphone and camera. Therefore, FORCEDENTRY’s impact was not only arbitrary code execution, but spyware-based monitoring of the victim’s device. [9] FORCEDENTRY was invisible to the victim because the malicious attachment was processed automatically by iMessage in the background. The victim did not need to click a link, open a file, or approve anything. However, the attack was not completely invisible from a forensic perspective. Investigators could still identify traces left on the device, such as suspicious attachment files, process crashes, network activity, and database artifacts. Files that appeared to have a .gif extension were found in the iPhone’s SMS attachments folder, but they were actually Adobe PDF files containing JBIG2-encoded data. Therefore, the exploit was invisible at the user-interface level, but it could still leave technical traces that made the compromise detectable during forensic analysis.
- **2022 exploit chains (iOS 15–16)**: Three disparate exploit chains (LATENTIMAGE, FINDMYPWN, and PWNYOURHOME) weaponized various flaws in iOS 15 and iOS 16 as zero-days to penetrate devices and launch Pegasus. FINDMYPWN used a two-phase exploit combining the Find My service and iMessage [10]. FINDMYPWN’s attack sequence appeared to begin with abnormal activity in the fmfd process, which is associated with Apple’s Find My feature, as the process exited and relaunched. A suspicious unacceptedShares item is being written and later deleted from a cache directory related to the Find My app. After this, iMessage related-activity appears where MessageBlastDoorService launches and relaunches.

PWNYOURHOME also used a two-phase exploit combining the HomeKit functionality which manages smart home devices and iMessage. The exploit could succeed even if the target never configured a "Home" inside HomeKit. In the first phase, logs showed the homed process crashing while decoding unusual HomeKit-related data. Some traces were left in the HomeKit database, especially in cases where Pegasus was unable to delete the attacker's Gmail address. In one case, this email address was added to HomeKit around eight minutes before Pegasus started running on the phone. In the second phase, the phone downloaded PNG images through iMessage and processing these images caused crashes in MessageBlastDoorService. This trace made the attack sequence clearer: the exploitation started with HomeKit, then continued with iMessage and finally resulted in Pegasus running on the phone. LATENTIMAGE appeared to leave very few traces on the device which made its full attack path difficult to reconstruct. Since the fmfd process exited and reloaded during exploitation, this indicated that this exploit may also have used Apple's Find My feature. However, Citizen Lab could not confirm whether Find My was the initial attack vector. Unlike FINDMYPWN and PWNYOURHOME, LATENTIMAGE launched the Pegasus spyware via springboard which proves it was a different exploit chain. [11]

- **BLASTPASS (2023):** NSO Group zero-click exploit chain discovered by Citizen Lab on the iPhone of a person working for a Washington DC-based civil society organization. The attack was capable of compromising iPhones running iOS 16.6, which was the latest iOS version at the time, without any interaction from the victim. The exploit used Passkit attachments containing malicious images sent from an attacker-controlled account. This means the victim did not need to click a link or manually open a malicious file for the exploit chain to be triggered. [12]The BLASTPASS attack happened as follows: the attacker sent an iMessage from an attacker-controlled account containing a PassKit attachment with malicious images. When the victim's iPhone received the message, iOS automatically processed the PassKit attachment and its malicious image content. This automatic processing triggered vulnerable iOS components, allowing arbitrary code execution. The malicious image was not installed on the device, it acted as the trigger for the exploit chain. After code execution was achieved, Pegasus spyware could be delivered and installed on the victim's iPhone.
- **QuaDream's REIGN / ENDOFDAYS (2021–2023):** Instead of iMessage, this exploit leveraged invisible iCloud calendar invitations to infect devices. The operator quietly added a calendar event invite for the target using the target's Apple ID [13].
- **WhatsApp CVE-2019-3568:** It is a vulnerability in WhatsApp's VoIP stack. It allowed remote code execution via a specially crafted series of RTP packets sent to a target phone number. The attack abused the way

WhatsApp automatically handled VoIP signaling data. [14] In other words, the attacker could send malicious call-related packets through WhatsApp's calling feature. Since WhatsApp processes these packets automatically when a call reaches the phone, the victim did not need to click a link or open a file. If the malformed RTCP packets triggered the buffer overflow successfully, attacker-controlled code could run on the device. [14]

4.1.2 Media and System Parser Processing Attacks

- CVE-2026-0006: a remote code execution vulnerability affecting Android's System Component. It involves an out of bounds read and write issue in multiple locations due to a heap buffer overflow. This means that when Android processes specially crafted input, the vulnerable code may read from or write to memory outside the intended heap buffer. If successfully exploited, this memory corruption could lead to remote code execution with no additional execution privileges and no user interaction needed. [15] This vulnerability affects Android's built-in media-processing layer rather than a normal user-installed application. It is related to Media Codecs component which are Android system components used to decode and process media such as audio, video, and media streams. Therefore, if specially crafted media-related input reaches this vulnerable processing path, Android may trigger a heap buffer overflow, which could lead to remote code execution without user interaction. In other words, the victim does not need to click a link, manually open a file, or grant permission for the vulnerability to be triggered. [16] The attack starts by delivering specifically crafted input over the network to the affected Android device. If this input reaches the vulnerable Android System/Media Codecs processing path, the vulnerable code may perform out-of-bounds read or write operations. This causes heap memory corruption. If the memory corruption is successfully controlled by the attacker, it could lead to remote code execution. Exploitation does not require additional execution privileges or user interaction, meaning that the victim does not need to click a link, open a file manually, or grant permission for the vulnerability to be triggered.

4.1.3 Wireless ADB authentication-bypass attacks:

This type of attack targets Android's wireless debugging service, known as Android Debug Bridge (ADB). CVE-2026-0073 is a critical Android remote code execution vulnerability affecting Android's ADB daemon, called `adbd`. The vulnerable code is located in `adbd_tls_verify_cert` in `auth.cpp`. The function `adbd_tls_verify_cert` is responsible for verifying the client's certificate/key during wireless ADB authentication. The vulnerability is

caused by a logic error in wireless ADB authentication, meaning that Android may make the wrong decision when checking whether a connecting ADB client is trusted. Normally, wireless ADB should only allow debugging access after verifying that the connecting client is authorized. However, because of the vulnerable certificate or key verification logic, an attacker with proximal or adjacent network access may bypass this authentication step if wireless ADB service is reachable. The issue was related to incorrect use of EVP_PKEY_{cmp} , because not all non-zero return values mean that the client key matches successfully. If exploitation succeeds, the attacker may obtain remote code execution as the Android shell user, with no additional execution privileges needed and no user interaction required. Therefore, the attack does not require the victim to click a link or open a file, but it does require the vulnerable wireless ADB service to be reachable from the attacker's adjacent network position. The vulnerability works at the Android system-service and authentication-logic level, with impact reaching the ADB shell user execution level. Public sources describe execution context as the shell user rather than full root compromise. [17] [18] [19]

4.2 Email, Office, and Collaboration Systems Abusing Automatic Processing

Zero-click attacks are not limited to mobile and consumer devices, they can also target email systems.

- **Office and Document Rendering:** CVE-2026-40361 targets Microsoft Office Word and related Office components. It is a use-after-free vulnerability that may allow an unauthorized attacker to execute code locally on the victim's machine. The vulnerability is classified as CWE-416 which refers to a use-after-free memory-management weakness. It has a CVSS v3.1 base score of 8.4, rated High, with the vector value UI:N. This means that no user interaction is required for exploitation. Therefore, CVE-2026-40361 is relevant to zero-click attack discussions because its CVSS vector includes UI:N, meaning that exploitation does not require user interaction in the scored scenario. In the context, "execute code locally" means that the malicious code would run on the victim's own computer, inside the affected Microsoft Office environment. [20] Based on the CWE-416 technique explanation, the attacker prepares specially crafted Office or Word content that is processed by Microsoft Word. While word is parsing and rendering this content, it may create internal objects in memory to handle document structures such as formatting, embedded elements, or other Office data. Due to the vulnerability, one of these memory objects may be freed while Word still keeps a reference to it. Later, Word may try to use the same freed memory again. This is called a dangling pointer situation. If the attacker can try to influence what data is placed in that freed memory area, Word may process attacker-controlled data as if it were still a

valid internal object. This can corrupt memory and may allow the attacker to execute code on the victim's machine inside the Microsoft Office Word environment. Therefore, the attack does not target the Word document as the final objective. The document is only the delivery container. The real target is the vulnerable Microsoft Office Word memory-handling logic that processes the document content. [21] and Windows OLE CVE-2025-21298: OLE, Object Linking Embedding is a Microsoft technology that allows documents and other content to be embed and link objects. The vulnerability can be exploited when an attacker sends a specially crafted mail to the victim. The exploitation may occur when the victim opens the malicious email using an affected version of Microsoft Outlook software or when Outlook displays a preview of the specially crafted email. [22]

- **Authentication Coercion** (This means that the attacker tricks the victim's machine to automatically trying to log in to an attacker-controlled server: Microsoft Outlook CVE-2023-23397 (2023): Microsoft Outlook for Windows vulnerability where an attacker exploits a weakness by delivering a specially crafted Outlook messages, calendar items, or tasks to a user. This item includes the PidLidReminderFileParameter extended MAPI property, which contains a UNC path pointing to a threat actor-controlled SMB server. When Outlook processes the reminder, the victim's window machine may automatically connect to the external SMB server and leak the user's Net-NTLMv2 hash. The attacker can later attempt to use for relay attacks or offline password cracking [23].
- **Email Client Processing:** Apple Mail CVE-2020-9818/CVE-2020-9819: Affected by vulnerabilities related to maliciously crafted email messages. CVE-2020-9819 could lead to heap corruption, while CVE-2020-9818 could cause unexpected memory modification or application termination. [24]

4.3 Telecom (SS7 Vulnerabilities)

SS7-based attacks are the quintessential "zero-click" attack vector in the telecom world, the victim simply needs a phone number. SS7 vulnerabilities usually comes from the way the protocol was originally designed. It was built for a trusted telecom environment where operator trusted each other. At that time, SS7 did not include strong built-in security controls. However, with the expansion of telecommunication networks, the number of providers, interfaces, and access points increased. As a result, SS7 became more vulnerable, not only because of its old design, but also because the telecom ecosystem became larger, more interconnected, and more open to abuse. This weakness allows the attackers with access to the SS7 signaling network to misuse normal telecom function. Instead of compromising the victim's phone directly, they can intercept messages, redirect calls, track the user's location, or support banking fraud. [25]

- **Call & SMS interception:** Attackers can reroute calls and text messages

through their own servers by exploiting SS7's routing functions. This is especially dangerous for SMS-based Two-Factor Authentication (2FA), by redirecting SMS messages containing 2FA codes, attackers can gain access to bank accounts or email accounts without the victim's knowledge [26].

- **Fraudulent transactions:** By manipulating SS7 signaling, attackers can reroute calls or messages meant for financial institutions or mobile banking apps, enabling them to initiate or approve fraudulent transactions and drain victims' accounts [27].
- **German bank heist (2017):** Cybercriminals exploited SS7 vulnerabilities to intercept 2FA codes sent to German bank customers. They first obtained banking credentials via phishing, then leveraged SS7 access to redirect SMS OTPs, authorizing fraudulent transfers resulting in substantial financial losses. This demonstrated the practical exploitation of SS7's **SendRoutingInfoForSM** operation to bypass SMS authentication [28].
- **2024 European banking attack:** A sophisticated cybercrime group intercepted SMS messages from thousands of banking customers across Europe, draining accounts of millions of euros within hours, without exploiting any software vulnerability or installing malware, relying solely on weaknesses in SS7 [29]. Global losses from SS7-enabled fraud were estimated at \$38.95 billion in 2023, and rogue operators continue to profit from selling illicit SS7 access on black markets [30].

4.4 AI Agents: The New Frontier

EchoLeak (CVE-2025-32711): Dubbed the first known zero-click attack on an AI agent, this vulnerability in Microsoft 365 Copilot could allow hackers to automatically exfiltrate the most sensitive information from Copilot's context without requiring any user interaction. The attack exploited what researchers call an "LLM scope violation," in which untrusted input from outside an organization could commandeer the AI model to access and steal privileged data [31].

Without any user interaction, an attacker's email could coerce Copilot into accessing internal files and transmitting their contents to an attacker-controlled server. The attack worked by injecting hidden prompt instructions into an external email that Copilot would automatically process when summarizing an inbox [32]. The emergence of Generative AI has birthed a new class of AI-native zero-click attacks. Recent vulnerabilities like EchoLeak demonstrate that GenAI assistants can be hijacked through "semantic exploits" that require no traditional code execution [33].

- **MCP (Model Context Protocol) :** A standard that connects AI assistants to the systems where data lives. It enables developers to build secure, two-way connections between their data sources and AI-powered tools.

Developers can either expose their data through MCP servers or build AI applications, called MCP clients that connect to these servers. [34] There can be vulnerabilities in MCP-enabled systems: - Tool poisoning : vulnerability where an attacker embeds malicious instructions inside the descriptions of MCP tools. In a MCP server, every tool has metadata associated with it, such as a name and a description. The AI model, also called LLM (Large Language Model which is the type of AI model that is trained on large amounts and can understand, generate, summarize, and reason over language) uses this metadata to decide what each tool does and decide which tools should be used for a given user request. If the tool description is compromised, the model may be manipulated into selecting unintended tools or bypassing security controls. These malicious instructions in tool metadata are invisible to users but they can be interpreted by the AI model and its underlying systems. This risk is especially important in hosted MCP server scenarios, where tool definitions may change after a user has already approved the tool. It is called “rug pull” by some researchers which means that a tool may appear safe at first, but later its description or behavior can be modified to include malicious instructions. As a result, a previously trusted tool could perform actions that were not originally declared, such as leaking sensitive data or manipulating information. [35]

- CVE-2026-33654: attacks the email channel processing module of a personal AI assistant called nanobot, specifically nanobot/channels/email.py. Before version 0.1.6, a remote unauthenticated attacker could send an email containing malicious LLM instructions to the bot’s monitored email address, the bot automatically polls, ingests, and processes the email content as highly trusted input without any interaction of the bot owner. This happens because the bot treats external email content as trusted instructions instead of untrusted data. As a result, the attacker could execute arbitrary LLM instructions and subsequently system tools, creating a stealthy zero-click attack. Version 0.1.6 patches the issue. [36]

4.5 IoT and Ecosystem-Level Wireless Attacks

- AirPlay “AirBorne” (2025): Zero-click risk is not limited to phones, messages, or email clients. It also applies to wireless protocols and SDKs used across many connected devices. Several AirPlay vulnerabilities, including CVE-2025-24252 (use-after-free) and CVE-2025-24132 (stack-based buffer overflow), were found to be “wormable.” Once an attacker compromises a vulnerable Airplay-enabled device, malware could autonomously scan for and infect other AirPlay-enabled devices on the same network, enabling rapid automated lateral movement through enterprise or home net-

works. AirBorne is not a classic supply-chain attack, because the attacker did not compromise Apple’s update system or distribute a malicious update. Instead, the risk came from the fact that AirPlay technology is used beyond Apple devices, including in third-party products such as smart TVs, speakers, receivers, and CarPlay systems. [37] Apple patched its own devices and released updated SDK and plug-in components to manufacturers. These manufacturers still needed to integrate the fixes and update their own products, which created a long-term patching problem because many smart-home and IoT devices may be updated slowly or may remain unpatched. [38] In a zero-click scenario, an attacker on the same local network could send malicious AirPlay traffic to a vulnerable AirPlay-enabled device. Since AirPlay-enabled devices process AirPlay traffic as part of their normal operation, the victim would not need to click a link, open a file, or manually accept anything for the vulnerable code path to be reached. [37] This makes AirBorne important because one vulnerable shared protocol or SDK can expose many different products across homes and enterprises.

- BlueBorne: It is an ecosystem-level wireless zero-click attack. It is a set of nine exploitable Bluetooth vulnerabilities that could allow an attacker of a targeted device to take complete control of a device and its data. It impacted almost every connected device running Android, Linux, Windows, and iOS versions before iOS 10, regardless of the Bluetooth version in use. BlueBorne used Bluetooth as the attack vector, so the attacker only needed to be within Bluetooth range of a vulnerable device with Bluetooth enabled. The victim did not need to click anything, open a file, pair with the attacker, or even set the device to discoverable mode. This makes BlueBorne relevant to zero-click attacks because the vulnerable device automatically listens for and processes Bluetooth traffic as part of its normal operation. BlueBorne is also important from an ecosystem perspective because Bluetooth is used across many device types, including phones, computers, and IoT devices. [39]

4.6 Enterprise / Server-Side / Network Infrastructure

Attacks targeting enterprise-specific technologies skyrocketed, accounting for 44% of all zero-day exploits in 2024, with a laser focus on security and networking products like VPNs and firewalls. Mandiant’s M-Trends 2025 confirms that for the fifth consecutive year, exploits are the 1 initial infection vector, responsible for 33% of all breaches investigated [40]. Attackers are increasingly targeting VPN gateways, firewall appliances, identity access systems, secure remote access infrastructure, and network edge devices, and then using compromised service accounts to move lat-

erally and evade authentication monitoring [41]. Attackers are no longer focusing only on phones, browsers, or email users especially because these targets have become harder to exploit due to stronger security protections. Instead, they are increasingly targeting enterprise technologies. In 2025, 43 zero-days representing 48 % of exploited zero-days, affected enterprise technologies. [42].

4.6.1 Server-Side Zero-Click Attacks Abusing Automatic Processing

- CVE-2026-28289 is a remote code execution vulnerability in FreeScout, a free help desk and shared inbox built with PHP'S Laravel framework. It is a patch bypass for CVE-2026-27636 and affects FreeScout 1.8.206 and earlier. This means that an earlier vulnerability had already been fixed, but the fix was incomplete, allowing attackers to use a different technique to bypass the protection and still reach remote code execution. The vulnerability allows an authenticated user with file-upload permissions to achieve RCE by uploading a malicious .htaccess file using a zero-width space character before its name. The vulnerable function is sanitizeUploadedFileName() in app/Http/Helper.php. The core problem is a Time-of-Check to Time-of-Use (TOCTOU) flaw in the filename validation process. This means that FreeScout checks the filename at one moment, but later uses a modified version of that filename after sanitization. In this case, FreeScout checks whether the uploaded filename begins with a dot before it removes invisible characters from the filename. The attacker can upload a file named: [zero-width space]. htaccess. [43]. Because the zero-width space is invisible, the filename visually looks like .htaccess, but technically it does not start directly with a dot during the first check. As a result, the file passes the security check. Later, FreeScout removes the invisible zero-width space during sanitization, so the filename becomes .htaccess and is saved on the server. This is dangerous because .htaccess files can affect how Apache handles files in a directory, which can enable remote code execution on vulnerable deployments. [44] This is considered a bypass because the security check looks at the filename too early. During validation, FreeScout checks [zero-width space].htaccess, which does not technically begin with a dot. However, after sanitization, the server uses .htaccess, which is the dangerous filename that the check was supposed to block. In other words, the attacker makes the file look safe during the check, but dangerous when it is actually saved and used [NVD/SentinelOne]. This is dangerous because .htaccess files can affect how Apache handles files in a directory, which can enable remote code execution on vulnerable deployments. [43] If exploitation succeeds, the attacker may execute arbitrary system commands on the FreeScout server. This could enable full server or system takeover,

access to stored emails, conversations, attachments, helpdesk tickets, and mailbox content, as well as data exfiltration, service disruption, and possible lateral movement to other systems in the same network. Therefore, the impact is on the FreeScout server and the sensitive support data stored or processed by it, not directly on a user’s personal laptop or phone.

- CVE-2026-27493 is a second-order expression injection vulnerability affecting n8n, an open-source workflow automation platform. The vulnerable component is the Form node processing logic, especially in workflows where a public form accepts unauthenticated user input and later displays or interpolates that same input inside another Form node field. The attack is called “second-order” because the malicious input is not necessarily executed immediately when it is submitted. Instead, the attacker first submits specially crafted form data that is initially accepted as normal user input. Later, when the workflow reuses this value, n8n may mistakenly interpret it as an expression instead of plain text. The vulnerable logic is related to the way n8n handles values that begin with the “=” character, because n8n expressions are normally identified using this symbol. Therefore, the data is accepted once as form input, then evaluated again later as n8n expression logic. By itself, this allows the attacker to inject and evaluate arbitrary n8n expressions within the workflow context and potentially access data available to that workflow. However, CVE-2026-27493 does not automatically give full control of the server by itself. In the first stage, the attacker can make n8n evaluate malicious expressions inside the workflow context. Normally, these expressions should be restricted by n8n’s sandbox, which is designed to prevent workflow expressions from directly executing operating-system commands. The attack becomes more dangerous if the attacker chains this expression injection with a separate sandbox escape vulnerability. In that case, the attacker can break out of the restricted expression environment and execute code on the actual machine or container running n8n. Once code execution is achieved on the host, the attacker may be able to compromise the n8n instance, read sensitive workflow data, access stored credentials such as API keys or OAuth tokens, and abuse the external services connected to n8n, such as cloud platforms, databases, email accounts, or SaaS applications. The attack does not directly infect a user’s laptop or phone; it works against the server-side n8n environment. It begins at the network level because the attacker sends data to an exposed public form endpoint, but the real weakness is at the application logic and expression-evaluation level. Only if the vulnerability is successfully chained with a sandbox escape does the attack move toward the host/server operating-system level, resulting in possible remote code execution on the n8n host. This exploitation also de-

depends on a specific workflow configuration: a Form node must reuse unauthenticated user input, the value must begin with “=” sign, and remote code execution requires a separate sandbox escape. [45] [46]
 NB: SaaS : Software as a Service: It means a software we use through the internet instead of installing and managing everything ourselves (ex. Gmail/Microsoft 365/Github...). API key is like a password for an application. OAuth token is also a type of access secret, but it usually comes from a user granting permission Expression injection: lets the attacker run something inside n8n’s expression environment. Sandbox escape: another vulnerability that lets the attacker escape that limited environment and reach the real server.

5 Mitigation Strategies

5.1 For Mobile & Endpoint

Organizations must adopt risk-based patching, prioritize actively exploited vulnerabilities, implement zero-trust architectures that limit lateral movement, deploy behavioral analytics to detect post-compromise activities, and enable platform-specific protections, such as iOS Lockdown Mode, for high-risk users. Patching velocity is critical: the five-day exploitation window demands automated, immediate update mechanisms [47]. The Cybersecurity and Infrastructure Security Agency (CISA) also supports this way of prioritizing patches through its Known Exploited Vulnerabilities (KEV) Catalog. This catalog lists vulnerabilities that attackers have already used in real attacks, helping organizations decide which security flaws should be fixed first. [48]

- Patching and version upgrades: Correct vulnerable code paths in device, mobile, or client-side components. In CVE-2026-0073, the mitigation works by correcting the flawed wireless ADB authentication logic. CVE-2026-0073 exists in *adb_tls_verify_cert* in *auth.cpp*, where a logic error could allow a bypass of wireless ADB mutual authentication. Since the attack depends on abusing this trust-checking process, the patch addresses the known authentication bypass and prevents an attacker from being treated as an authorized ADB client. In addition to applying the Android security update, users and administrators should disable Wireless Debugging when it is not needed. This reduces exposure because the vulnerability targets Android’s wireless ADB authentication path, which is relevant when wireless ADB is enabled or reachable by a nearby or adjacent attacker. [18] [17] In CVE-2026-40361, the mitigation is to apply Microsoft’s security update for the affected Office and Word products. The vulnerability is a use-after-free issue in Microsoft Office that could allow an unauthorized attacker to execute code locally. Since use-after-free is a

memory bug where a program may incorrectly use memory after it has already been released, this vulnerability can be understood as a memory-management or memory-safety issue. [20] The update fixes the vulnerable memory-management condition, reducing the risk that crafted Office content can trigger code execution through the affected component.

- Attack-Surface Reduction: Reducing the number of exposed features, services, interfaces, or processing paths that an attacker can reach. CVE-2026-0073 Android Wireless ADB disables wireless Debugging by reducing the exposure to the wireless ADB authentication path.

5.2 For Telecom (SS7)

The shift in mitigation strategy has moved from a perimeter-based, trust-centric model to a multi-layered, behavioral security approach, incorporating machine learning-based SS7 firewalls and Zero Trust principles. Defense must move beyond filtering known malicious Global Titles to continuous monitoring for anomalous behavior and continuous verification of all entities. On the user side, replacing SMS-based 2FA with app-based authenticators (TOTP) or hardware keys eliminates the most common SS7 attack surface [49].

5.3 For Email

- CVE-2023-23397 was actively exploited by the nation-state group Forest Blizzard to obtain secret, unauthorized access to email accounts within exchange servers. Microsoft recommends installing Outlook update as soon as possible. Microsoft defender XDR can also help detect known exploitation attempts and post-compromise activity. If patching is not immediately possible, organisation should ensure to add users to the Protected Users Group to prevent NTLM from being used as an authentication mechanism and can also block outbound TCP port 445/SMB traffic through perimeter firewall, local firewall, and through their VPN settings. [50]
- Organisations should also scan Exchange mailboxes for suspicious messages, calendar items, or tasks containing the "PidLidReminder-FileParameter" property. Microsoft provides a script that identify these items and remove either the malicious property or the malicious property or the entire message.
- Organisations using an on-premises Microsoft Exchange Server should apply the latest security updates to ensure that defense-in-depth mitigations are active. If suspicious or malicious reminder values are detected, they should make sure to use the script to remove either

the messages or just the properties, and consider initiating incident response activities. For targeted or compromised users, organizations should reset the passwords of accounts that were logged in to the affected computers. Microsoft also recommends using multifactor authentication to reduce the impact of potential Net-NTLMv2 relay attacks, although it does not prevent authentication data from being leaked or cracked offline. Other measures include disabling unnecessary Exchange services, restricting inbound connections on ports 135 and 445 to approved addresses and disabling NTLM where possible. The Outlook security update remains necessary regardless of where the organization's email is hosted. [50]

5.4 For AI Systems

Enforcing a hard separation between untrusted and trusted content in the LLM's prompt context is a key defense. Content originating from outside (e.g., external emails) should never be concatenated inline with a user's internal documents and query. One approach is content source tagging: wrapping external text with special tokens or annotations so the model can distinguish its provenance [31]. arXiv Microsoft also recommends DLP tags to block processing of external emails, and a new M365 feature that restricts Copilot from accessing emails labeled with sensitivity tags.

Prompt injection as a risk where crafted input manipulates the model's behavior and may cause it to ignore instructions, disclose sensitive information, or perform unauthorized actions. Indirect prompt injection occurs when malicious instructions are hidden inside external content, such as websites, documents, emails, files, or tool outputs, that the AI system processes later. Instead of attacking the user directly, the attacker places instructions in content that the model is expected to read. When the model interprets this content, it may mistakenly treat the hidden text as a legitimate instruction, which can change the model's behavior and lead to unintended actions such as leaking private conversation content, exposing sensitive data, or triggering unauthorized tool use. [51] So untrusted content may be used as information, but it must never gain authority over the AI system. Based on this principle, the following mitigation strategies focus on preventing untrusted content from gaining authority over the AI system while still allowing the AI to use it safely as information.

- Content source tagging is a mitigation approach where external or third-party content given to the AI is clearly marked according to its source or trust level. This helps the AI system distinguish trusted instructions from lower-trust content such as emails, webpages, retrieved documents, uploaded files, or tool outputs. However, source tagging must be implemented carefully: the source label should be assigned by the system, not by the content itself. This mitigation can be implemented through techniques such as spotlighting which is designed to defend against indirect

prompt injection. It uses data marking and metaprompting to isolate and neutralize external content within prompts and is useful when AI applications process third-party documents that may contain embedded malicious instructions, especially user-uploaded files or external web content. [52] This should be complemented by system-controlled provenance and metadata integrity. Provenance and metadata should be protected across the RAG pipeline by hashing documents at ingestion, verifying hashes before retrieval, tracking who uploaded each document and whether it was approved, storing access-control metadata with retrieved chunks, enforcing access checks at retrieval time, and using signed source attribution with verifiable document hashes. [53] These controls help ensure that the system verifies where the content came from, who uploaded it, whether it was approved, what access permissions apply to it, and whether it has been modified. As a result, a document, email, webpage, or retrieved file should not be able to declare itself “trusted,” “internal,” or “approved”; the system should assign and protect those trust labels.

- Prompt partitioning means that the AI system should not combine instructions and data into one flat prompt where they appear to have the same authority. Attackers take advantage of the fact that the AI can mix up commands and data when both are written in natural language. To mitigate this risk, trusted system instructions, user requests, external content, retrieved documents, and tool outputs should be placed into clearly separated sections. The prompt should be structured in a way that clearly separates trusted instructions from user data and external content. Lower-trust content, such as emails, web pages, retrieved documents, or tool outputs, should be treated as data to analyze, not as commands to follow. This reduces the risk that attacker-controlled content is interpreted as a legitimate instruction by the AI agent. [54]
- Input/Output filtering means checking both the input before the AI processes it and the output before it leaves the AI system. Input filtering may detect hidden prompt-injection instructions, attempts to override system rules, suspicious formatting, encoded instructions, malicious tool-use requests and hidden text in documents or webpages. Output filtering may prevent sensitive data leakage, unauthorized tool calls, unsafe instructions, external exfiltration, policy violations, and disclosure of internal prompts or secrets. [51] [54] [55]

6 Raiha

6.1 Telecommunications Zero-Click Attack Surfaces

Telecommunications systems continuously receive, decode, validate, route, and act on signalling, radio, voice, media, and messaging traffic without requiring subscriber interaction. Network registration, roaming updates, SMS deliv-

ery, SIP requests, VoLTE and Wi-Fi Calling session setup, GTP-U forwarding, PFCP session control, and radio-layer frames are processed automatically to maintain service availability [25]. The attack surface therefore extends across SS7, Diameter, IMS, SIP and VoLTE, SMS infrastructure, roaming and inter-working interfaces, 4G/5G core-network functions, management services, and mobile baseband processors.

Zero-click exposure occurs when attacker-controlled structured input reaches a privileged telecommunications component and is processed before a subscriber or administrator can observe, approve, or reject the operation.

6.1.1 Automatic Processing and Telecom Trust Boundaries

Telecommunications networks cross multiple trust boundaries, including mobile devices, radio access networks, carrier cores, IMS components, messaging systems, interconnect providers, roaming partners, and management networks. Messages are frequently processed according to the apparent trust of the sending network function or peer.

SS7 illustrates the risk created by broad inter-operator trust. An entity with signalling access may misuse legitimate operations affecting subscriber location, SMS routing, call delivery, or service availability [25]. Diameter introduces stronger controls but still depends on correct peer authorisation, message filtering, routing, roaming policy, and protocol-state validation.

The central risk is therefore not protocol age alone. Legacy and modern telecom systems become zero-click-relevant when automatically processed input crosses a trusted interface without sufficient origin validation, authorisation, protocol-state enforcement, parser safety, or isolation.

6.1.2 IMS, SIP, VoLTE, and Wi-Fi Calling Processing

IMS uses SIP and related media-description protocols to establish, modify, and terminate voice and multimedia sessions. Receiving components automatically parse request methods, headers, identities, routing fields, sequence numbers, authentication values, Session Description Protocol attributes, and media parameters before completing registration, authorisation, routing, and session state evaluation.

Vulnerabilities may arise from malformed or duplicated fields, inconsistent normalisation, unsafe length handling, oversized values, optional-parameter processing, invalid state transitions, or memory-unsafe parsing. VoLTE and Wi-Fi Calling inherit this exposure because subscriber lookup, SIP signalling, media negotiation, and baseband processing occur automatically, including during incoming communication.

The expanded dataset contains four Samsung Exynos baseband CVEs in which malformed SDP attributes delivered through IMS-related processing could reach privileged modem code without user interaction, demonstrating a remote Internet-to-baseband attack path through carrier-supported VoLTE or Wi-Fi Calling services.

Interworking gateways increase exposure by translating messages, identities, routing information, and session state between LTE, 5G, legacy mobile networks, circuit-switched systems, and external providers.

6.1.3 SMS and Messaging Infrastructure

SMS traffic is stored, routed, filtered, forwarded, and delivered through carrier and device components before it is displayed to the user. Carrier systems process message content, sender and recipient identifiers, encoding, routing, delivery status, and binary or concatenated-message metadata. Device-side components may invoke additional parsers for media, multipart, binary, or specially formatted content.

Signalling abuse and parser exploitation must be treated separately. Signalling abuse manipulates legitimate routing or delivery functions, whereas parser exploitation targets implementation flaws in carrier services, basebands, operating systems, or messaging applications [14, 26, 27].

6.1.4 Baseband and Radio-Layer Processing

The mobile baseband processes radio-layer frames, registration, mobility, handover, media data, and communication with the application processor. Because this processing occurs below the mobile operating system, malformed radio, signalling, or media traffic may reach memory-unsafe or incorrectly validated firmware without user interaction.

Baseband vulnerabilities represent the strongest telecom zero-click alignment in the dataset because input is processed automatically, the vulnerable code executes below the operating system, and conventional endpoint-security tools have limited visibility into the affected processor.

Detection therefore depends mainly on indirect evidence such as modem crashes, unexpected modem or device restarts, temporary service loss, abnormal registration behaviour, radio anomalies, and vendor diagnostic logs. Strong isolation between the baseband and application processor is necessary to limit cross-boundary impact.

6.1.5 4G/5G Core-Network Processing

The expanded dataset also includes vulnerabilities in Open5GS, an open-source 4G/5G core-network implementation. These cases affected GTP-U and PFCP processing in user-plane and session-control functions. GTP-U carries subscriber user traffic between radio and core-network components, while PFCP controls forwarding sessions between control-plane and user-plane functions.

Malformed packets reaching these interfaces may trigger memory corruption, invalid array access, or service failure before authentication at the application level. Practical exploitation normally requires access to a RAN, roaming, interconnect, or internal core-network path rather than direct public Internet exposure.

These CVEs show that zero-click research must include internal carrier trust boundaries: a service does not need to be publicly exposed for automatically processed attacker-controlled traffic to reach a critical core function.

6.1.6 Telecommunications CVE Findings

The telecommunications component of the expanded CVE intelligence base contains 18 vulnerabilities: 11 rated High and 7 rated Medium for zero-click relevance. The dataset includes 11 Critical, 5 High, and 2 CVEs with no vendor-published numeric CVSS severity. CVSS severity and zero-click relevance are treated separately: CVSS estimates technical severity, while the research classification evaluates automatic processing, authentication state, interaction requirements, reachability, and trust-boundary impact.

Parser and message-handling failures were the dominant root cause, accounting for 7 of 18 records, followed by memory-safety failures with 6 of 18 records. Together, these categories account for 13 of the 18 telecom CVEs and show that unsafe handling of structured protocol data is the most consistent technical pattern in the dataset.

Only 3 of the 18 telecom CVEs were recorded as known exploited, and 2 were included in CISA KEV. Zero-click relevance therefore identifies an automatic, no-interaction attack path; it must not be presented as proof that exploitation has occurred in the wild.

The findings map to six telecommunications attack-surface groups:

- **Baseband and radio-layer processing — 9 CVEs.** Seven Samsung or MediaTek records and two Qualcomm records affected baseband, modem data-network, or multimode call-processing components. The vulnerabilities involved RLC/LLC reassembly, CSN.1 packet parsing, SDP attribute handling, RTP/NALU media reassembly, and mobility information processing. The Samsung CVE-2023-41111 and CVE-2023-41112 chain demonstrates radio-adjacent baseband code execution through a rogue base station, while CVE-2023-24033 and CVE-2023-26496–26498 demonstrate Internet-to-baseband exposure through IMS-related media-description parsing [56–59].
- **VoIP and call-processing stacks — 1 CVE, High.** WhatsApp CVE-2019-3568 affected inbound call-processing and SRTP handling before the recipient answered. The vulnerability was exploited in real attacks, showing that automatically processed communications traffic can create a remote zero-click delivery path into a user device [60].
- **Carrier and edge-management interfaces — 2 CVEs.** Cisco IOS XE CVE-2023-20198 enabled unauthenticated creation of a privilege-15 account through the Web UI, while CVE-2023-20273 enabled command execution after access was established. The pair demonstrates how an automatically processed unauthenticated management request can become

the first stage of a chained compromise affecting carrier-grade routing, switching, or wireless infrastructure [61].

- **VoIP endpoints, PBX systems, and wireless controllers — 3 CVEs, all Medium zero-click relevance.** These cases involved unauthenticated management or web processing but had narrower practical reachability. Exposure depended on enterprise network position, voice or management VLAN access, a non-default configuration, or an administrative-access outcome rather than a direct parser-to-code- execution path [62–64].
- **Device-side messaging and media parsers — 1 CVE, Medium.** Android Stagefright CVE-2015-3864 involved automatic media processing, but practical exposure depended on operating-system version, messaging application behaviour, and automatic retrieval configuration [65].
- **4G/5G core-network protocol processing — 2 CVEs, Medium.** Open5GS CVE-2023-23846 and CVE-2021-41794 affected GTP-U and PFCP handling. Both were automatically processed before subscriber-level authentication, but exploitation required access to a trusted or reachable core-network interface. Their inclusion expands the research beyond endpoint and Internet-facing vulnerabilities to internal RAN-to-core and control-plane-to-user-plane trust boundaries [66, 67].

Across the dataset, the recurring attack path was consistent: attacker-controlled radio, signalling, media, management, or core-network input reached a parser, protocol handler, authentication workflow, or management service; the component processed that input automatically; and the resulting failure affected code execution, authentication, administrative control, service availability, or a sensitive telecommunications trust boundary.

A High zero-click relevance classification does not mean exploitation is possible against every deployment. Practical conditions may include radio proximity, Internet exposure, network position, affected hardware, software or firmware version, enabled features, IMS reachability, or vendor and carrier configuration. High relevance instead reflects strong alignment with the research conditions: automatic processing, no victim interaction, pre-authentication or unauthenticated handling, remote or adjacent reachability, and impact on a sensitive device, signalling, session, core-network, or management boundary.

The dataset still contains no direct SS7 or Diameter implementation CVE. SS7 and Diameter remain evidence-supported architectural attack surfaces, but they should not be presented as CVE clusters demonstrated by the current dataset. Additional telecom-specific CVE collection is required to close this evidence gap.

6.1.7 Telecom Mitigation and Remediation

Mitigation differs across the identified attack-surface groups:

- **Baseband and radio-layer processing** — deploy chipset, device-vendor, OEM, and carrier firmware updates; maintain accurate modem and firmware inventories; verify that corrected builds have reached affected devices; disable 2G, VoLTE, or Wi-Fi Calling temporarily where the vendor recommends doing so and operational requirements permit [56–59].
- **VoIP and call-processing stacks** — apply vendor application or communications-stack updates and perform compromise assessment for devices suspected of targeting rather than treating patching as sufficient [60].
- **Carrier and edge-management interfaces** — apply fixed software releases, remove public HTTP or HTTPS management exposure, restrict administration to trusted networks, and hunt for unauthorised accounts, configuration changes, command-execution artefacts, and implants [61].
- **VoIP endpoints, PBX systems, and wireless controllers** — upgrade firmware, segment voice and management networks, restrict embedded web services, remove risky non-default configurations, and rotate administrative credentials when access may have been obtained [62–64].
- **Device-side messaging and media parsers** — apply platform patches and disable automatic retrieval of complex message media where operationally possible [65].
- **4G/5G core-network functions** — upgrade affected Open5GS versions, restrict GTP-U and PFCP interfaces to authorised RAN, SMF, roaming, and interconnect peers, and enforce segmentation, firewalling, or IPsec protection on internal carrier interfaces [66,67].

6.1.8 Telecom Mitigation Gap Review

The mitigation-gap review found that **vendor mitigation was available for all 18 telecom CVEs, but a clearly identified fixed version or build was recorded for only 7 of 18**. Temporary workarounds were available for 14 records, while the remaining four were unclear or dependent on vendor and deployment conditions.

Credential or token rotation was explicitly addressed for only 1 of 18 CVEs; session invalidation and rebuild or factory-reset guidance were not explicitly addressed for any of the 18 records. Compromise assessment was addressed for only 3 of 18. Residual mitigation gaps were assessed as Moderate for 12 cases and Minor for 6.

The most significant gaps were concentrated in the highest-risk attack surfaces:

- **Baseband and radio-layer processing.** The existence of a chipset-vendor fix did not always give defenders a clearly identifiable corrected build at device level. Baseband patches may pass through chipset vendors, device manufacturers, OEMs, carriers, regional firmware channels, and device-update mechanisms.

An upstream baseband fix therefore does not prove that an affected handset or modem has received, installed, and is executing the corrected firmware.

Advisories also provided limited post-compromise guidance. Defenders were not consistently given procedures for validating modem integrity, determining whether arbitrary baseband code had executed, collecting reliable forensic evidence, or restoring a trusted baseband state. This creates the most significant residual gap because defensive visibility is weakest where the vulnerable component is most privileged [56–59].

- **VoIP and call-processing stacks.** The application update corrected the vulnerable path, but confirmed real-world exploitation means previously exposed devices may require broader forensic review.

The remaining gap is not the absence of a patch, but limited vendor guidance for determining whether a device was already compromised and whether attacker persistence, account access, or spyware remains after the update [60].

- **Carrier and edge-management interfaces.** Fixed releases and configuration mitigations were comparatively clear, but patching did not remove attacker-created accounts, configuration changes, or possible persistence established before remediation.

Operational recovery therefore requires threat hunting, account review, configuration validation, credential rotation, and verification of a trusted post-remediation state in addition to patch deployment [61].

- **4G/5G core-network functions.** Version upgrades and interface restrictions were available, but the residual risk depends heavily on whether GTP-U and PFCP interfaces are correctly segmented and limited to authorised peers. Core-network interfaces are often treated as trusted internal paths, which can reduce inspection and logging precisely where malformed protocol traffic may be automatically processed.

The mitigation gap is therefore partly architectural: applying a software fix does not correct excessive peer trust, weak segmentation, or insufficient protocol-aware monitoring between RAN, control-plane, user-plane, and roaming components [66, 67].

- **VoIP endpoints, PBX systems, wireless controllers, and messaging parsers.** These cases produced mainly Minor residual gaps because fixed versions, affected configurations, or practical workarounds were more clearly identifiable. However, remediation still depended on accurate asset inventory, firmware distribution, configuration validation, and removal of exposed management paths [62–65].

The central finding is the difference between patch availability and recovery readiness. A vendor fix changes vulnerable code, but does not by itself prove

complete deployment, invalidate attacker access, remove persistence, establish device or modem integrity, or return the system to a trusted state.

Future defensive work should therefore focus on proving the following: the affected version has been replaced across all relevant devices and nodes; temporary exposure has been removed; unauthorised access and persistence have been investigated; credentials or sessions have been invalidated where applicable; and observable evidence confirms that the corrected system rejects or safely handles the previously dangerous input path.

6.1.9 Telecom Detection and Defensive Controls

- **Baseband and radio-layer processing** — maintain modem, chipset, device, firmware, and carrier-build inventories; deploy vendor updates; monitor modem crashes, radio-service restarts, registration anomalies, unexpected service loss, and rogue-base-station indicators; and enforce isolation between baseband and application-processor components [56–59].
- **IMS, SIP, VoLTE, and call-processing stacks** — apply SIP, SDP, SRTP, and media grammar validation; normalise headers and attributes; enforce message-size and state limits; and monitor call setup, parser failures, modem or service crashes, and abnormal incoming-session patterns [58, 60].
- **Carrier and edge-management interfaces** — remove public management exposure, restrict access to administrative networks, centralise logs, and monitor for new accounts, privilege changes, configuration drift, command execution, unusual Web UI requests, and implant indicators [61].
- **4G/5G core-network functions** — restrict GTP-U and PFCP interfaces to authorised peers, validate protocol fields and state, centralise UPF/SMF logs, and monitor malformed extension headers, abnormal session-establishment messages, process crashes, and repeated service restarts [66, 67].
- **VoIP endpoints, PBX systems, and wireless controllers** — segment voice and management networks, restrict embedded web interfaces, review non-default configurations, and monitor abnormal management traffic, authentication changes, and service instability [62–64].
- **Device-side messaging and media parsers** — limit automatic content retrieval, treat binary and multipart content as untrusted, and use fuzz testing, compiler hardening, memory-safe implementation techniques, and parser isolation [65].
- **SS7 and Diameter signalling** — validate message origin, subscriber relationship, roaming agreement, requested operation, peer authorisation, and protocol state; monitor abnormal location queries, routing changes, roaming activity, and signalling volume [25].

Telecommunications traffic must not be trusted solely because it arrived through an established carrier, roaming, radio, voice, core-network, or management path. The source, purpose, protocol state, subscriber context, requested operation, peer authorisation, and affected trust boundary should be validated before input is allowed to influence routing, identity, mobility, administration, forwarding, or session state [25].

6.2 VPN and Remote-Access Zero-Click Attack Surfaces

VPN and remote-access appliances are high-value zero-click targets because they are commonly Internet-facing and must automatically process TLS, HTTP, IPSec, IKE, authentication, session, portal, API, and management traffic before trust is established. The same appliance may operate as a protocol parser, authentication broker, reverse proxy, tunnel endpoint, session manager, policy enforcement point, and administrative platform. A single pre-authentication flaw can therefore compromise both the gateway and the enterprise trust boundary it is intended to protect [68–70].

The central architectural risk is that a privileged, Internet-facing appliance must parse and act on attacker-controlled input before authentication can succeed or fail.

6.2.1 Pre-Authentication and Automatic Processing

VPN gateways cannot authenticate a requester without first accepting and processing network traffic. TLS negotiation, HTTP routing, cookies, request headers, URL paths, form data, WebSocket messages, IKE payloads, SAML responses, and product-specific protocol fields may therefore reach complex code before identity is established.

This early processing creates a direct path to memory corruption, path traversal, command injection, request smuggling, server-side request forgery, SQL injection, authentication bypass, or information disclosure. No employee needs to click a link or approve a connection when the vulnerable service automatically processes the request [70, 71].

6.2.2 SSL VPN, TLS, HTTP, IPSec, and IKE Processing

SSL VPN services parse TLS negotiation data and HTTP paths, parameters, headers, cookies, bodies, rewritten URLs, and portal resources. Incorrect length handling, unsafe normalisation, inadequate bounds checking, or unsanitised input may produce memory corruption, traversal, disclosure, or injection [68, 71].

IPSec gateways parse chained IKE payloads before a secure tunnel exists. Malformed lengths, fragmentation, sequencing, state transitions, and cryptographic negotiation fields may produce out-of-bounds access, heap corruption, denial of service, or code execution [69, 72, 73].

Clientless portals create an additional parsing layer because the gateway may rewrite links, proxy internal applications, fetch remote resources, transform

content, and maintain user sessions. A defect in this path can expose internal files, credentials, memory, or session material without requiring a valid VPN login.

6.2.3 Authentication, Identity, and Session Boundaries

VPN appliances commonly integrate with SAML, OIDC, LDAP, RADIUS, MFA, directories, device-posture services, and local administrator databases. Incorrect signature verification, claim mapping, issuer or audience validation, role assignment, token handling, or access-control enforcement may convert invalid identity data into an authenticated or privileged session.

Authentication bypass is especially important in zero-click research because the vulnerable workflow itself may create access without valid credentials. However, successful access after bypass remains distinct from memory-safety exploitation: one crosses an identity boundary, while the other may directly change process execution.

Patching an authentication flaw does not automatically revoke sessions, cookies, tokens, certificates, API keys, or administrative accounts created before remediation.

6.2.4 Management Planes, APIs, and Reverse Proxies

Administrative web interfaces, REST APIs, diagnostics, configuration backup, certificate management, routing controls, logging functions, and firmware operations expand the attack surface beyond the VPN tunnel itself. These interfaces frequently execute with high privilege and may be exposed through the same Internet-facing appliance.

Reverse proxies and backend request handlers can also create indirect reach. Request smuggling or server-side request forgery may allow an attacker to reach internal management functions that are not directly exposed. Command injection or arbitrary file creation may then convert limited request-processing access into operating-system execution.

6.2.5 Patch Exposure, High Availability, and Visibility

VPN gateways are difficult to patch because they are availability-critical and often remain exposed while organisations prepare change windows. High availability clusters introduce an additional risk: all active and standby nodes, recovery images, templates, and replacement appliances must be updated and validated.

Proprietary appliances frequently provide less endpoint visibility than general-purpose servers. Compromise may therefore appear only through parser errors, crashes, restarts, unusual requests, configuration changes, new accounts, certificate replacement, abnormal sessions, or unexplained outbound connections [71].

6.2.6 VPN and Remote-Access CVE Findings

The current VPN and remote-access component of the CVE intelligence base contains **37 vulnerabilities: 19 rated High, 17 Medium, and 1 Low for zero-click relevance**. The dataset contains **29 Critical and 8 High CVSS-severity vulnerabilities**.

Thirty-one of the 37 CVEs were recorded as known exploited, and 30 were included in the CISA Known Exploited Vulnerabilities catalogue. This concentration demonstrates that Internet-facing remote-access infrastructure is not only theoretically exposed but repeatedly targeted in operational attacks.

Zero-click relevance and exploitation status remain separate measures. A CVE may have a strong automatic pre-authentication path without confirmed exploitation, while another may be widely exploited even though practical access depends on a specific configuration or chained condition.

The root-cause distribution was:

- **Memory-safety failures — 14 CVEs.** These included stack and heap overflows, out-of-bounds reads and writes, use-after-free conditions, and format-string weaknesses. They affected SSL VPN daemons, remote-desktop services, IPSec handlers, application-delivery controllers, and clientless portals.
- **Input-validation and injection failures — 12 CVEs.** Nine records were classified as input-validation failures, two as injection flaws, and one as unsafe command or code execution. Outcomes included path traversal, SQL injection, command injection, SSRF, and arbitrary file creation leading to command execution.
- **Authentication and authorisation failures — 11 CVEs.** Seven records involved authentication logic errors or flaws, while four involved authorisation or access-control failures. These vulnerabilities allowed unauthenticated access, privilege escalation, session abuse, information disclosure, or access to protected management functions.

Memory-safety failure remained the largest single root-cause category, accounting for 14 of 37 records. However, authentication, authorisation, input-validation, and injection weaknesses together accounted for 23 of 37, showing that VPN compromise is not primarily a memory-corruption problem.

The findings map to the following operational attack surfaces:

- **Pre-authentication SSL VPN and secure-access gateways.** Fortinet, Ivanti, Citrix, Palo Alto Networks, SonicWall, Check Point, and Pulse Secure products exposed automatically processed SSL VPN, portal, WebSocket, or gateway traffic before a trusted session existed. Relevant examples include CVE-2024-21762, CVE-2023-3519, CVE-2025-0282, CVE-2025-22457, CVE-2022-42475, CVE-2023-27997, CVE-2019-19781, CVE-2019-11510, and CVE-2023-4966 [74–82].

- **IPSec and IKE processing.** Ivanti CVE-2024-21894 affected pre-authentication IPSec processing and demonstrated that packet-level heap corruption may be reachable before a VPN tunnel or authenticated user session exists [83].
- **Authentication, identity, and access-control workflows.** The dataset includes bypasses or access-control failures affecting ConnectWise Screen-Connect, Fortinet administrative interfaces, Citrix ADC and Gateway, Sophos Firewall, Palo Alto Networks SAML processing, Ivanti secure-access products, and Cisco VPN routers. These cases show that automatic identity processing can cross the gateway boundary even without a parser-to-code-execution path [84–90].
- **Clientless portals and information disclosure.** Check Point CVE-2024-24919, Cisco CVE-2020-3259, Fortinet CVE-2018-13379, Pulse Secure CVE-2019-11510, and Citrix CVE-2023-4966 demonstrate that unauthenticated file or memory disclosure can expose credentials, session tokens, configuration, or sensitive internal information without immediately executing code [81, 82, 91–93].
- **Management-plane, API, and backend-processing interfaces.** F5 iControl REST, F5 TMUI, Fortinet management services, Cisco VPN-router management interfaces, Ivanti CSA, FortiClientEMS, and Palo Alto Networks GlobalProtect management-related processing exposed request smuggling, SSRF, SQL injection, authentication bypass, command injection, or arbitrary file creation [94–101].
- **Remote-desktop services and gateways.** Microsoft CVE-2019-1181 and CVE-2020-0609 affected pre-authentication remote-desktop processing. These records extend the research beyond dedicated VPN appliances to remote-access services that expose a comparable privileged perimeter [102, 103].

The recurring pattern was consistent: an Internet-facing or remotely reachable service automatically processed unauthenticated or early-authentication input, and a parser, portal, identity, session, API, or management flaw then crossed the gateway trust boundary.

The findings span multiple vendors, appliance generations, and remote-access technologies. The concentration is therefore best explained by the privileged architectural role and exposure of these products rather than by one vendor alone.

A High zero-click relevance classification does not imply universal exploitability. Practical exposure may depend on software version, enabled services, network configuration, public reachability, authentication state, support status, or a required exploit chain. High relevance instead indicates strong alignment with automatic processing, no victim interaction, pre-authentication or unauthenticated reachability, and impact on a privileged remote-access boundary.

6.2.7 VPN Mitigation and Remediation

- **Pre-authentication SSL VPN and portal flaws** — apply fixed releases through an emergency process; disable or restrict affected services where patching is delayed; update all HA nodes, templates, and recovery images; and investigate exposed systems for evidence of prior compromise [74–79].
- **IPSec and IKE processing** — upgrade affected gateways, restrict unnecessary exposure, review negotiation errors and crash logs, rotate pre-shared keys where compromise is suspected, and validate peer configuration [83].
- **Authentication and identity boundaries** — patch, rotate credentials and API secrets, invalidate sessions, enforce phishing-resistant MFA, review local and federated accounts, and validate SAML, OIDC, LDAP, RADIUS, issuer, audience, claim, and role mappings [84, 86–89].
- **Clientless portals and information disclosure** — patch, identify files or memory exposed by the flaw, rotate credentials, password hashes, private keys, certificates, cookies, and tokens that may have been disclosed, and revoke affected sessions [81, 82, 91–93].
- **Management planes and APIs** — remove public exposure, restrict administration to trusted hosts or dedicated networks, patch, and hunt for rogue administrators, keys, configuration changes, web shells, command-execution artefacts, disabled logging, and altered routes [94–99, 101].
- **Remote-desktop services** — apply security updates, restrict public RDP and RD Gateway exposure, use Network Level Authentication where supported, and review crash and connection telemetry [102, 103].
- **End-of-life products** — isolate or disable exposed services as an immediate measure and replace unsupported appliances rather than treating a workaround as a permanent security state.

6.2.8 VPN Mitigation Gap Review

The current mitigation-gap review found that **vendor mitigation was available for 36 of 37 VPN CVEs, while a clearly identified fixed version or build was recorded for 34 of 37**. Temporary workarounds were available for 19 records, unavailable for one, and unclear or undocumented for 17.

Credential or token rotation was addressed for 30 of 37 records, but session invalidation was explicitly addressed for only 4 of 37. Compromise assessment was addressed for 23, and rebuild or factory-reset guidance for only 10.

Detection guidance was available for all 37 records, but detection advice did not necessarily establish whether compromise had occurred or whether the appliance had returned to a trusted state.

Residual mitigation gaps were assessed as **27 Moderate**, **5 Severe**, **3 Significant**, and **2 Minor**. Four records raised an end-of-life concern.

- **Patch deployment does not restore trust.** A fixed release may correct the vulnerable code while leaving active sessions, stolen cookies, certificates, local users, API keys, web shells, modified configuration, or attacker persistence intact.

The largest documented recovery gap is session hygiene: fixed builds were clearly identified for 34 CVEs, but session invalidation was explicitly addressed for only 4.

- **Pre-authentication gateway compromise.** Memory-corruption or command-execution vulnerabilities may provide access at the appliance operating-system level. In these cases, credential rotation alone is insufficient unless defenders also assess persistence, system integrity, configuration changes, and internal movement.
- **Disclosure requires secret-specific remediation.** Arbitrary file-read and memory-disclosure vulnerabilities may expose passwords, hashes, private keys, certificates, cookies, and session tokens. Remediation must identify what data could have been read and rotate or revoke each affected secret class.
- **Authentication fixes require identity cleanup.** Authentication bypass may create valid-looking sessions or privileged accounts. Defenders must review local administrators, federated identities, group mappings, MFA registrations, active tokens, and access logs rather than relying solely on the updated authentication check.
- **High-availability and recovery assets.** Patching the active gateway does not prove that standby nodes, snapshots, templates, recovery images, or replacement appliances are corrected. Reintroduction of an older image can restore the vulnerable state.
- **End-of-life appliances.** Four records raised an end-of-life concern. Unsupported Internet-facing gateways may lack a durable vendor fix and should be replaced, with isolation or service disablement used only as an interim control [90, 98, 99, 104].

The central mitigation gap is the difference between applying a patch and proving recovery. The trusted state is not restored until vulnerable nodes are updated, attacker access is revoked, persistence is investigated, identity and configuration are validated, and clean operation is confirmed through evidence.

6.2.9 VPN Detection Opportunities

- **Pre-authentication parsing** — monitor malformed HTTP, TLS, IKE, WebSocket, and portal requests; parser errors; heap or process faults;

unexplained crashes; service restarts; and repeated requests to unusual paths [71, 74, 83].

- **Authentication and identity** — monitor invalid SAML or OIDC issuers, audiences, signatures, claims, and assertion timing; unexpected MFA registration; local-account creation; privilege changes; abnormal LDAP or RADIUS mappings; and sessions without a corresponding authentication event [71].
- **Sessions** — detect impossible travel, unusual source changes, concurrent use, token reuse, abnormal duration, sessions surviving account disablement or password reset, and activity after expected revocation [105].
- **Clientless portals and disclosure** — monitor traversal sequences, unauthenticated file requests, abnormal memory or diagnostic endpoints, bulk configuration access, and requests for credential or session stores [81, 91–93].
- **Management planes and APIs** — alert on new administrators, API-key creation, certificate replacement, changed routes, altered group mappings, disabled logging, configuration exports, shell commands, web files, and outbound connections from the appliance [71, 105].
- **Availability and integrity signals** — correlate crashes, restarts, HA failovers, configuration drift, integrity-check failures, and unexplained resource consumption with inbound request telemetry.

An unexplained crash, restart, or HA failover on an Internet-facing gateway should be investigated as a potential security signal, not treated solely as a reliability event.

6.2.10 VPN Defensive Controls

Internet-facing and KEV-listed VPN vulnerabilities should enter an emergency remediation process when exploitation is known [71, 106].

Unused portals, APIs, diagnostics, management interfaces, protocols, and legacy VPN modes should be disabled. Administrative services should be removed from public exposure and restricted to dedicated trusted networks.

TLS and IKE configurations should disable obsolete protocols and weak algorithms, validate certificates strictly, rotate pre-shared keys, and enforce payload, sequence, replay, state, and fragmentation checks [69, 107].

Phishing-resistant MFA should be enforced but must not be treated as mitigation for pre-authentication flaws. SAML, OIDC, LDAP, and RADIUS integrations should apply strict issuer, audience, signature, claim, time, and least-privilege role validation.

Sessions, cookies, and tokens should be short-lived, securely bound, centrally revocable, and invalidated after account changes, suspicious activity, exposure to

disclosure vulnerabilities, or confirmed compromise. VPN address pools should be treated as untrusted network zones with restricted internal reachability.

Logs should be centralised outside the appliance and monitored for crashes, restarts, administrator changes, configuration drift, certificate changes, unusual portal activity, identity anomalies, and abnormal sessions. External logging is important because local logs may be altered or destroyed after gateway compromise.

High-availability peers, golden images, templates, backups, and recovery appliances should be included in patch verification. A rollback should never reintroduce a vulnerable version.

Where exploitation is suspected, patching alone is insufficient. Defenders should isolate the appliance, preserve available evidence, revoke sessions, rotate credentials and keys, review identity integrations, hunt for persistence and internal movement, and restore from a trusted configuration or image when integrity cannot be established.

Unsupported Internet-facing gateways should be replaced rather than represented as permanently mitigated systems.

The central defensive principle is that a VPN gateway must be treated as a privileged Internet-facing security boundary that continuously processes hostile input and therefore requires rapid remediation, minimal exposed functionality, external monitoring, strong identity controls, and verified post-compromise recovery [68, 69, 71].

References

- [1] R. Wood. (2025, Oct.) Zero-click cyber attacks & AI agents. TimeTrex. See section: Case Study: The Pegasus Saga and NSO Group’s Arsenal. [Online]. Available: <https://www.timetrex.com/blog/zero-click-cyber-attacks>
- [2] S. Chourasiya, G. Samanta, D. K. Sardar, P. Sharma, and C. V. Kumar, “Pegasus spyware: A vulnerable behaviour-based attack system,” in *2023 2nd International Conference on Edge Computing and Applications (ICECAA)*, 2023, pp. 287–292.
- [3] B. Marczak, J. Scott-Railton, N. Aljizawi, S. Anstis, and R. Deibert, “The great ipwn: Journalists hacked with suspected NSO Group iMessage ‘zero-click’ exploit,” The Citizen Lab, University of Toronto, Tech. Rep. 135, Dec. 2020. [Online]. Available: <https://citizenlab.ca/research/the-great-ipwn-journalists-hacked-with-suspected-nso-group-imessage-zero-click-exploit/>
- [4] Apple. (2024, May) BlastDoor for Messages and IDS. Apple Platform Security. [Online]. Available: <https://support.apple.com/en-lb/guide/security/secdd3c881cee/web>

- [5] S. Groß. (2021, Jan.) A look at iMessage in iOS 14. Google Project Zero. [Online]. Available: <https://projectzero.google/2021/01/a-look-at-imessage-in-ios-14.html>
- [6] B. Marczak, J. Scott-Railton, B. A. Razzak, N. Al-Jizawi, S. Anstis, K. Berdan, and R. Deibert, “Forcedentry: Nso group imessage zero-click exploit captured in the wild,” The Citizen Lab, University of Toronto, Research Report 143, Sep. 2021. [Online]. Available: <https://citizenlab.ca/research/forcedentry-nso-group-imessage-zero-click-exploit-captured-in-the-wild/>
- [7] TimeTrex. (2025, Oct.) Zero-click cyber attacks & ai agents. Published October 6, 2025. Accessed: 2026-06-08. [Online]. Available: <https://www.timetrex.com/blog/zero-click-cyber-attacks>
- [8] R. Naraine. (2021, Dec.) Google says nso pegasus zero-click “most technically sophisticated exploit ever seen”. SecurityWeek. Accessed: 2026-06-08. [Online]. Available: <https://www.securityweek.com/google-says-nso-pegasus-zero-click-most-technically-sophisticated-exploit-ever-seen/>
- [9] Amnesty International. (2021, Jul.) Massive data leak reveals israeli NSO Group’s spyware used to target activists, journalists, and political leaders globally. Amnesty International. [Online]. Available: <https://www.amnesty.org/en/latest/press-release/2021/07/the-pegasus-project/>
- [10] R. Lakshmanan. (2023, Apr.) Nso group used 3 zero-click iphone exploits against human rights defenders. The Hacker News. [Online]. Available: <https://thehackernews.com/2023/04/nso-group-used-3-zero-click-iphone.html>
- [11] B. Marczak, J. Scott-Railton, B. Abdul Razzak, and R. Deibert, “Triple threat: Nso group’s pegasus spyware returns in 2022 with a trio of ios 15 and ios 16 zero-click exploit chains,” The Citizen Lab, University of Toronto, Tech. Rep. 165, Apr. 2023. [Online]. Available: <https://citizenlab.ca/research/nso-groups-pegasus-spyware-returns-in-2022/>
- [12] The Citizen Lab. (2023, Sep.) BLASTPASS: NSO Group iphone zero-click, zero-day exploit captured in the wild. The Citizen Lab, University of Toronto. [Online]. Available: <https://citizenlab.ca/blastpass-nso-group-iphone-zero-click-zero-day-exploit-captured-in-the-wild/>
- [13] A. Roy, “Technical research report: Advanced iphone zero-click exploits and spyware (2019 - 2025),” 06 2025.
- [14] National Vulnerability Database. (2019, May) Cve-2019-3568 detail. National Institute of Standards and Technology. WhatsApp VOIP Stack Buffer Overflow Vulnerability. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2019-3568>

- [15] ——. (2026, Mar.) CVE-2026-0006 Detail. National Institute of Standards and Technology. Android heap buffer overflow vulnerability allowing remote code execution without user interaction. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2026-0006>
- [16] Android Open Source Project. (2026, Mar.) Android security bulletin—march 2026. Google. Updated May 28, 2026; includes CVE-2026-0006. [Online]. Available: <https://source.android.com/docs/security/bulletin/2026/2026-03-01>
- [17] National Vulnerability Database. (2026, May) CVE-2026-0073 Detail. National Institute of Standards and Technology. Android wireless ADB mutual authentication bypass vulnerability allowing adjacent remote code execution without user interaction. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2026-0073>
- [18] Android Open Source Project. (2026, May) Android security bulletin—may 2026. Google. Updated May 7, 2026; includes CVE-2026-0073. [Online]. Available: <https://source.android.com/docs/security/bulletin/2026/2026-05-01>
- [19] J. Duong. (2026, Mar.) Fix EVP_PKEY_cmp usage. Android Open Source Project. Commit 842d331f9e5fb10770a09379e19240daea057dba; affects daemon/auth.cpp. [Online]. Available: <https://android.googlesource.com/platform/packages/modules/adb/+842d331f9e5fb10770a09379e19240daea057dba>
- [20] National Vulnerability Database. (2026, May) CVE-2026-40361 Detail. National Institute of Standards and Technology. Microsoft Office use-after-free vulnerability allowing local code execution. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2026-40361>
- [21] MITRE. (2026) CWE-416: Use After Free. Common Weakness Enumeration. CWE Version 4.20. [Online]. Available: <https://cwe.mitre.org/data/definitions/416.html>
- [22] Microsoft Security Response Center. (2025, Jan.) Windows ole remote code execution vulnerability. Microsoft. CVE-2025-21298. [Online]. Available: <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2025-21298>
- [23] Microsoft Incident Response, “Guidance for investigating attacks using cve-2023-23397,” <https://www.microsoft.com/en-us/security/blog/2023/03/24/guidance-for-investigating-attacks-using-cve-2023-23397/>, Mar. 2023, microsoft Security Blog. Accessed: 2026-06-08.
- [24] Apple. (2020, May) About the security content of ios 13.5 and ipados 13.5. Apple Support. [Online]. Available: <https://support.apple.com/en-ug/103029>

- [25] K. Ullah, I. Rashid, H. Afzal, M. M. W. Iqbal, Y. A. Bangash, and H. Abbas, “Ss7 vulnerabilities—a survey and implementation of machine learning vs rule based filtering for detection of ss7 network attacks,” *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 1337–1371, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/8984216>
- [26] SOCRadar. (2024, Oct.) Why ss7 attacks are the biggest threat to mobile security: Exploiting global telecom networks. Accessed: 2026-06-08. [Online]. Available: <https://socradar.io/blog/why-ss7-attacks-are-the-biggest-threat-to-mobile-security-exploiting-global-telecom-networks/>
- [27] P1 Security. (2024, Oct.) Understanding ss7 attacks: Vulnerabilities, impacts, and protection measures. [Online]. Available: <https://www.p1sec.com/blog/understanding-ss7-attacks-vulnerabilities-impacts-and-protection-measures>
- [28] Hackers-Arise. (2025, Jan.) Chinese state-sponsored hackers inside the u.s. mobile telecom system: Mobile telecom companies vulnerable to ss7 vulnerability. [Online]. Available: <https://hackers-arise.com/chinese-state-sponsored-hackers-inside-the-us-mobile-telecom-system-mobile-telecom-companies-vulnera>
- [29] Terrazone. (2025, Dec.) Ss7 security vulnerabilities: The complete guide to attacks and prevention. Published December 6, 2025. Accessed: 2026-06-08. [Online]. Available: <https://terrazone.io/ss7-security-vulnerabilities-attacks-prevention/>
- [30] R. Kumar. Ss7 vulnerabilities: How hackers exploit telecom networks. How2Lab. [Online]. Available: <https://www.how2lab.com/tech/mobile-communication/ss7-vulnerabilities>
- [31] D. Jones. (2025, Jun.) Critical flaw in microsoft copilot could have allowed zero-click attack. Cybersecurity Dive. Accessed: 2026-06-08. [Online]. Available: <https://www.cybersecuritydive.com/news/flaw-microsoft-copilot-zero-click-attack/750456/>
- [32] P. Reddy and A. S. Gujral, “Echoleak: The first real-world zero-click prompt injection exploit in a production llm system,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.10540>
- [33] E. Archibong, P. Asuquo, and B. Stephen, “Review of zero-click attacks and zero-trust security model: concepts, architecture, state of the art, and future directions,” *Discover Networks*, vol. 2, 04 2026.
- [34] Anthropic. (2024, Nov.) Introducing the model context protocol. Anthropic. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>
- [35] S. Young and D. Delimarsky. (2025, Apr.) Protecting against indirect prompt injection attacks in MCP. Microsoft for Developers. [Online]. Available: <https://developer.microsoft.com/blog/protecting-against-indirect-injection-attacks-mcp>

- [36] National Vulnerability Database. (2026, Mar.) CVE-2026-33654 Detail. National Institute of Standards and Technology. Nanobot indirect prompt injection vulnerability in the email channel processing module. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2026-33654>
- [37] U. Katz, G. Elbaz, and A. Lumelsky. (2025, Apr.) AirBorne: Wormable zero-click remote code execution (RCE) in AirPlay protocol puts Apple & IoT devices at risk. Oligo Security. [Online]. Available: <https://www.oligo.security/blog/airborne>
- [38] Apple. (2025, Apr.) About the security content of AirPlay audio SDK 2.7.1, AirPlay video SDK 3.6.0.126, and CarPlay communication plug-in R18.1 updates. Apple Support. Includes CVE-2025-24132 and CVE-2025-30422. [Online]. Available: <https://support.apple.com/en-us/122403>
- [39] Armis. (2018, Sep.) BlueBorne: One year later. Armis. [Online]. Available: <https://www.armis.com/blog/blueborne-one-year-later/>
- [40] DeepStrike. (2025) Zero-day exploit statistics 2025: What defenders need. Accessed: 2026-06-08. [Online]. Available: <https://deepstrike.io/blog/zero-day-exploit-statistics-2025>
- [41] Ampcus Cyber Content Team. (2026, Mar.) Zero-day exploitation trends: What security leaders must prepare for in 2026. Accessed: 2026-06-08. [Online]. Available: <https://www.ampcuscyber.com/blogs/zero-day-exploitation-trends-what-security-leaders-must-prepare/>
- [42] C. Charrier, J. Sadowski, Z. Work, C. Lecigne, B. Sevens, and F. Plan. (2026, Mar.) Look what you made us patch: 2025 zero-days in review. Google Threat Intelligence Group. [Online]. Available: <https://cloud.google.com/blog/topics/threat-intelligence/2025-zero-day-review>
- [43] National Vulnerability Database. (2026, Mar.) CVE-2026-28289 Detail. National Institute of Standards and Technology. FreeScout patch bypass vulnerability allowing remote code execution through malicious file upload. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2026-28289>
- [44] SentinelOne. (2026, Mar.) CVE-2026-28289: Freescout rce vulnerability via bypass. SentinelOne Vulnerability Database. FreeScout patch bypass remote code execution vulnerability. [Online]. Available: <https://www.sentinelone.com/vulnerability-database/cve-2026-28289/>
- [45] National Vulnerability Database. (2026, Feb.) CVE-2026-27493 Detail. National Institute of Standards and Technology. N8n second-order expression injection vulnerability in Form nodes. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2026-27493>

- [46] SentinelOne. (2026, Feb.) CVE-2026-27493: n8n workflow automation rce vulnerability. SentinelOne Vulnerability Database. N8n second-order expression injection vulnerability in Form nodes. [Online]. Available: <https://www.sentinelone.com/vulnerability-database/cve-2026-27493/>
- [47] Cyber Security News, “One year of zero-click exploits: What 2025 taught us about modern malware,” <https://cybersecuritynews.com/one-year-of-zero-click-exploits/>, 2025.
- [48] Cybersecurity and Infrastructure Security Agency, “Cybersecurity and infrastructure security agency,” U.S. Department of Homeland Security, Fact Sheet, Nov. 2018. [Online]. Available: <https://www.cisa.gov/resources-tools/resources/cisa-fact-sheet>
- [49] R. Kumar. (2025) Ss7 vulnerabilities: How hackers exploit telecom networks. How2Lab. [Online]. Available: <https://www.how2lab.com/tech/mobile-communication/ss7-vulnerabilities>
- [50] Microsoft Incident Response. (2023, Mar.) Guidance for investigating attacks using CVE-2023-23397. Microsoft Security Blog. Updated February 15, 2024. [Online]. Available: <https://www.microsoft.com/en-us/security/blog/2023/03/24/guidance-for-investigating-attacks-using-cve-2023-23397/>
- [51] OWASP. (2025) LLM01:2025 Prompt Injection. OWASP Gen AI Security Project. [Online]. Available: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- [52] Microsoft. (2026, Mar.) Defend against indirect prompt injection attacks. Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/security/zero-trust/sfi/defend-indirect-prompt-injection>
- [53] OWASP Cheat Sheet Series. Retrieval-augmented generation (RAG) security cheat sheet. OWASP. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/RAG_Security_Cheat_Sheet.html
- [54] ——. LLM Prompt Injection Prevention Cheat Sheet. OWASP. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/LLM.Prompt_Injection_Prevention_Cheat_Sheet.html
- [55] Microsoft. (2026, Feb.) Prompt shields in microsoft foundry. Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/azure/foundry/openai/concepts/content-filter-prompt-shields>
- [56] TASZK Security Labs, “Samsung Baseband RLC Data Re-Assembly Buffer Overflow: CVE-2023-41111 and CVE-2023-41112,” <https://labs.taszk.io/blog/post/93-rlc.bof/>, 2024, accessed: 2026-07-24.

- [57] —, “MediaTek Baseband GPRS PNCD Heap Buffer Overflow: CVE-2022-21744,” https://labs.taszk.io/blog/post/84_mtk_pncd_bof/, 2023, accessed: 2026-07-24.
- [58] Google Project Zero, “Multiple Internet-to-Baseband Remote Code Execution Vulnerabilities in Exynos Modems,” <https://projectzero.google/2023/03/multiple-internet-to-baseband-rce.html>, 2023, discusses CVE-2023-24033 and related Exynos baseband vulnerabilities CVE-2023-26496, CVE-2023-26497, and CVE-2023-26498; Accessed: 2026-07-24.
- [59] Qualcomm, “September 2025 Security Bulletin,” <https://docs.qualcomm.com/product/publicresources/securitybulletin/september-2025-bulletin.html>, 2025, covers CVE-2025-21483 and CVE-2025-27034; Accessed: 2026-07-24.
- [60] Facebook, “CVE-2019-3568: WhatsApp VoIP Stack Buffer Overflow,” <https://www.facebook.com/security/advisories/cve-2019-3568>, 2019, accessed: 2026-07-24.
- [61] Cisco, “Multiple Vulnerabilities in Cisco IOS XE Software Web UI Feature,” <https://sec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-iosxe-webui-privesc-j22SaA4z>, 2023, covers CVE-2023-20198 and CVE-2023-20273; Accessed: 2026-07-24.
- [62] —, “Cisco IP Phones Web Server Remote Code Execution and Denial of Service Vulnerability,” <https://sec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-voip-phones-rce-dos-cRKPazlU>, 2020, accessed: 2026-07-24.
- [63] RedTeam Pentesting GmbH, “Auerswald COMpact Multiple Backdoors,” <https://www.redteam-pentesting.de/en/advisories/rt-sa-2021-004/-auerswald-compact-backdoor>, 2021, accessed: 2026-07-24.
- [64] Cisco, “Cisco Wireless LAN Controller Management Interface Authentication Bypass Vulnerability,” <https://sec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-wlc-auth-bypass-JRNhV4fF>, 2022, accessed: 2026-07-24.
- [65] Android Open Source Project, “Nexus Security Bulletin—October 2015,” <https://source.android.com/docs/security/bulletin/2015-10-01>, 2015, accessed: 2026-07-24.
- [66] GitHub Security Advisories, “Open5GS GTP Library Out-of-Bounds Write: CVE-2023-23846,” <https://github.com/advisories/GHSA-3vj7-j945-rq57>, 2023, accessed: 2026-07-24.
- [67] NCC Group, “Open5GS Stack Buffer Overflow During PFCP Session Establishment on UPF: CVE-2021-41794,” <https://www.nccgroup.com/research/>

technical-advisory-open5gs-stack-buffer-overflow-during-pfcp-session-establishment-on-upf-cve-2021-41721, accessed: 2026-07-24.

- [68] S. Frankel, P. Hoffman, A. Orebaugh, and R. Park, “Guide to ssl vpns,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-113, 2008, accessed: 2026-07-24. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/113/final>
- [69] E. Barker, Q. Dang, S. Frankel, K. Scarfone, and P. Wouters, “Guide to ipsec vpns,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-77 Revision 1, 2020, accessed: 2026-07-24. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/77/r1/final>
- [70] MITRE ATT&CK, “External Remote Services, Technique T1133,” <https://attack.mitre.org/techniques/T1133/>, 2026, accessed: 2026-07-24.
- [71] Cybersecurity and Infrastructure Security Agency, “Guidance and Strategies to Protect Network Edge Devices,” <https://www.cisa.gov/resources-tools/resources/guidance-and-strategies-protect-network-edge-devices>, 2025, accessed: 2026-07-24.
- [72] C. Kaufman, P. Hoffman, Y. Nir, P. Eronen, and T. Kivinen, “Internet key exchange protocol version 2 (ikev2),” Internet Engineering Task Force, Tech. Rep. RFC 7296, 2014, accessed: 2026-07-24. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7296>
- [73] V. Smyslov, “Internet key exchange protocol version 2 (ikev2) message fragmentation,” Internet Engineering Task Force, Tech. Rep. RFC 7383, 2014, accessed: 2026-07-24. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7383>
- [74] Fortinet, “FortiOS SSL VPN Out-of-Bounds Write: CVE-2024-21762,” <https://www.fortiguard.com/psirt/FG-IR-24-015>, 2024, accessed: 2026-07-24.
- [75] Citrix, “NetScaler ADC and NetScaler Gateway Remote Code Execution: CVE-2023-3519,” <https://support.citrix.com/article/CTX561482>, 2023, accessed: 2026-07-24.
- [76] Ivanti, “Security Advisory for CVE-2025-0282 and CVE-2025-0283,” <https://forums.ivanti.com/s/article/Security-Advisory-Ivanti-Connect-Secure-Policy-Secure-and-Neurons-for-ZTA-Gateways-CVE-2025-0282>, 2025, accessed: 2026-07-24.
- [77] —, “April Security Advisory: CVE-2025-22457,” <https://forums.ivanti.com/s/article/April-Security-Advisory-Ivanti-Connect-Secure-Policy-Secure-ZTA-Gateways-CVE-2025-22457>, 2025, accessed: 2026-07-24.

- [78] Fortinet, “FortiOS SSL VPN Heap-Based Buffer Overflow: CVE-2022-42475,” <https://www.fortiguard.com/psirt/FG-IR-22-398>, 2022, accessed: 2026-07-24.
- [79] —, “FortiOS and FortiProxy SSL VPN Heap-Based Buffer Overflow: CVE-2023-27997,” <https://www.fortiguard.com/psirt/FG-IR-23-097>, 2023, accessed: 2026-07-24.
- [80] Citrix, “Citrix ADC and Gateway Path Traversal: CVE-2019-19781,” <https://support.citrix.com/article/CTX267027>, 2019, accessed: 2026-07-24.
- [81] Ivanti, “Pulse Secure VPN Arbitrary File Reading: CVE-2019-11510,” <https://forums.ivanti.com/s/article/SA44101>, 2019, accessed: 2026-07-24.
- [82] Citrix, “NetScaler ADC and Gateway Sensitive Information Disclosure: CVE-2023-4966,” <https://support.citrix.com/article/CTX579459>, 2023, accessed: 2026-07-24.
- [83] Ivanti, “Security Advisory for CVE-2024-21894, CVE-2024-22052, CVE-2024-22053, and CVE-2024-22023,” <https://forums.ivanti.com/s/article/CVE-2024-21894-CVE-2024-22052-CVE-2024-22053-CVE-2024-22023>, 2024, accessed: 2026-07-24.
- [84] ConnectWise, “ScreenConnect Security Bulletin for CVE-2024-1709,” <https://www.connectwise.com/company/trust/security-bulletins/connectwise-screenconnect-23.9.8>, 2024, accessed: 2026-07-24.
- [85] Fortinet, “FortiOS, FortiProxy, and FortiSwitchManager Authentication Bypass: CVE-2022-40684,” <https://www.fortiguard.com/psirt/FG-IR-22-377>, 2022, accessed: 2026-07-24.
- [86] Citrix, “Citrix ADC and Citrix Gateway Authentication Bypass: CVE-2022-27510,” <https://support.citrix.com/article/CTX463706>, 2022, accessed: 2026-07-24.
- [87] Sophos, “Sophos Firewall Authentication Bypass and Remote Code Execution: CVE-2022-1040,” <https://www.sophos.com/en-us/security-advisories/sophos-sa-20220325-sfos-rce>, 2022, accessed: 2026-07-24.
- [88] Palo Alto Networks, “PAN-OS SAML Authentication Bypass: CVE-2020-2021,” <https://security.paloaltonetworks.com/CVE-2020-2021>, 2020, accessed: 2026-07-24.
- [89] Ivanti, “Connect Secure and Policy Secure Authentication Bypass: CVE-2023-46805,” <https://forums.ivanti.com/s/article/KB-CVE-2023-46805-Authentication-Bypass-CVE-2024-21887-Command-Injection-for-Ivanti-Connect-2024>, 2024, accessed: 2026-07-24.

- [90] Cisco, “Cisco Small Business RV Series Router Vulnerabilities: CVE-2023-20025,” <https://sec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-sbr042-multi-vuln-ej76Pke5>, 2023, accessed: 2026-07-24.
- [91] Check Point, “Security Gateway Information Disclosure Vulnerability: CVE-2024-24919,” <https://support.checkpoint.com/results/sk/sk182336>, 2024, accessed: 2026-07-24.
- [92] Cisco, “Cisco ASA and FTD WebVPN Information Disclosure: CVE-2020-3259,” <https://sec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-asaftd-info-disclose-9eJtycMB>, 2020, accessed: 2026-07-24.
- [93] Fortinet, “FortiOS SSL VPN Path Traversal: CVE-2018-13379,” <https://www.fortiguard.com/psirt/FG-IR-18-384>, 2019, accessed: 2026-07-24.
- [94] F5, “BIG-IP Configuration Utility Request Smuggling: CVE-2023-46747,” <https://my.f5.com/manage/s/article/K000137353>, 2023, accessed: 2026-07-24.
- [95] —, “BIG-IP and BIG-IQ iControl REST Authentication Bypass: CVE-2021-22986,” <https://my.f5.com/manage/s/article/K03009991>, 2021, accessed: 2026-07-24.
- [96] Fortinet, “FortiOS and FortiProxy Authentication Bypass: CVE-2024-55591,” <https://www.fortiguard.com/psirt/FG-IR-24-535>, 2025, accessed: 2026-07-24.
- [97] —, “FortiClientEMS SQL Injection: CVE-2023-48788,” <https://www.fortiguard.com/psirt/FG-IR-24-007>, 2024, accessed: 2026-07-24.
- [98] Cisco, “Cisco RV Series Routers Command Injection: CVE-2021-1609,” <https://sec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-rv34x-rce-8bfG3XkG>, 2021, accessed: 2026-07-24.
- [99] Ivanti, “Cloud Services Appliance Command Injection: CVE-2024-8190,” <https://forums.ivanti.com/s/article/Security-Advisory-Ivanti-CSA-4-6-Cloud-Services-Appliance-CVE-2024-8190>, 2024, accessed: 2026-07-24.
- [100] Fortinet, “FortiClientEMS SQL Injection: CVE-2023-48788,” <https://www.fortiguard.com/psirt/FG-IR-24-007>, 2024, accessed: 2026-07-24.
- [101] Palo Alto Networks, “PAN-OS GlobalProtect Command Injection: CVE-2024-3400,” <https://security.paloaltonetworks.com/CVE-2024-3400>, 2024, accessed: 2026-07-24.

- [102] Microsoft, “Remote Desktop Services Remote Code Execution Vulnerability: CVE-2019-1181,” <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2019-1181>, 2019, accessed: 2026-07-24.
- [103] —, “Windows Remote Desktop Gateway Remote Code Execution Vulnerability: CVE-2020-0609,” <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2020-0609>, 2020, accessed: 2026-07-24.
- [104] Cisco, “Cisco RV340, RV345, RV345P, and RV340W SSL VPN Buffer Overflow: CVE-2022-20699,” <https://sec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-sb-rv-overflow-s2r82P9v>, 2022, accessed: 2026-07-24.
- [105] MITRE ATT&CK, “Detection Guidance for External Remote Services, Technique T1133,” <https://attack.mitre.org/techniques/T1133/>, 2026, accessed: 2026-07-24.
- [106] Cybersecurity and Infrastructure Security Agency, “Known Exploited Vulnerabilities Catalog,” <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>, 2026, accessed: 2026-07-24.
- [107] Y. Nir, T. Kivinen, P. Wouters, and D. Migault, “Algorithm implementation requirements and usage guidance for ikev2,” Internet Engineering Task Force, Tech. Rep. RFC 8247, 2017, accessed: 2026-07-24. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8247>